



MODELING AND SIMULATION

Simulation & Modeling of a Shared Dormitory Laundry Room

Submitted to:

Prof. Modesto Tarrazona

Submitted by:

Annika Lois R. Dumalogdog

547692

Program:

BS Computer Science

Course Code:

CSE 10/L - 8263

Institution:

University of Mindanao - College of Computing Education

Date Submitted:

May 2026

Table of Contents

i. Cover page	1
ii. Table of Contents	2
I. Introduction	3
II. Overview of Modeling and Simulation	3
III. Types of Simulation Models	4
IV. Modeling Process	5
V. Simulation Tools and Techniques	8
VI. Case Study / Simulation Example A. Problem Description B. Objectives C. System Description D. Model Design E. Simulation Setup F. Assumptions	9
VII. Results & Analysis	14
VIII. Recommendation	15
IX. Conclusion	15

I. Introduction

Modeling and Simulation is a powerful analytical technique that builds a digital model of a real-world system, allowing engineers and decision-makers to experiment, test changes, and predict performance without disrupting actual operations. By executing the model under controlled stochastic conditions, simulation reveals system behavior that is often invisible to direct observation, especially when randomness, queueing, and resource contention are involved.

In service systems, simulation has become an essential decision-support tool. It helps managers evaluate trade-offs such as adding capacity, changing service policies, or redesigning physical layouts, all before committing real budget. The output provides data-driven evidence, including average wait times, utilization rates, throughput, and confidence intervals that quantify uncertainty.

This project focuses on a Laundry Queue System, a shared residential laundromat with multiple washing machines, time-varying customer arrivals, and the realistic possibility of users "hogging" machines after their wash cycle completes. Using a Discrete-Event Simulation (DES) engine built in TypeScript and visualized in real-time 3D, the study evaluates how machine count, queue capacity, and hogging behavior affect resident wait times and overall system performance.

II. Overview of Modeling and Simulation

Modeling is the process of creating an abstract representation of a real-world system, capturing only the essential features relevant to a specific question or objective. A model simplifies reality by focusing on key variables, relationships, and constraints while omitting unnecessary details.

Simulation is the dynamic execution of a model over time. It imitates the operation of a real-world process or system, often using random inputs to account for uncertainty. Simulation allows analysts to observe system behavior, measure performance metrics, and test alternative scenarios without interfering with the actual system.

The primary benefits of modeling and simulation include:

- Cost reduction: Testing policies virtually avoids capital expenditure on unproven solutions
- Risk mitigation: Identifying bottlenecks or failures before they occur in reality
- Time compression: Simulating weeks of operation in seconds or minutes
- Repeatability: Running the same scenario hundreds of times to obtain statistically valid results
- Insight generation: Revealing emergent behaviors that are not obvious from intuition alone

Simulation is widely used across industries. Manufacturing engineers simulate assembly lines to identify production bottlenecks. Healthcare administrators simulate emergency rooms to reduce patient waiting times. Logistics companies simulate supply chains to optimize inventory levels. In each case, simulation provides evidence-based guidance for decision-making.

This project applies discrete-event simulation (DES) to a shared dormitory laundry room. DES is specifically designed for systems where entities (residents) move through a sequence of activities (arriving, waiting, washing, leaving) and compete for limited resources (washing machines). The next section explores the different types of simulation models available to analysts.

III. Types of Simulation Models

Simulation models can be classified along several dimensions. Understanding these classifications helps the analyst select the appropriate modeling approach for a given problem.

The laundry room system is a dynamic model. The state changes continuously throughout the 16-hour operating day. Machine availability shifts from idle to busy to hogged. Queue lengths grow and shrink as residents arrive and depart. Resident positions move from arrival to waiting to washing to leaving. Capturing these changes over time is

essential to measuring wait times and utilization rates. A static model that only looked at a snapshot would completely miss the peak-hour congestion that is the core problem.

The simulation is a stochastic model. Three sources of randomness are built into the simulation: The resident arrival times, the wash cycle duration, and the hogging occurrence. Because of this randomness, running the simulation once is insufficient. The project uses 50 independent replications to obtain statistically valid estimates with 95% confidence intervals.

The laundry room system is a discrete model. Specifically, it is a discrete-event simulation (DES). Changes occur only at well-defined moments, where between these events, absolutely nothing happens. The simulation clock jumps from event to event, skipping idle time. This is what makes DES computationally efficient.

Thus, DES is the specific type used in this project. In DES, the simulation clock jumps from one event to the next, skipping periods of inactivity. This makes DES highly efficient for queueing systems like the laundry room.

IV. Modeling process

Building a valid and useful simulation model follows a structured methodology. This section describes the standard simulation modeling process and maps each step to the laundry room project.

Step 1: Problem Formulation

The analyst identifies the system to be studied, defines specific objectives, and establishes performance metrics. Without a clear problem statement, the simulation risks answering the wrong question.

Laundry room application: The problem is excessive resident waiting times due to peak-hour congestion and machine hogging. Objectives are to measure average wait time and balking rate. Metrics include minutes waited, machine utilization, and percentage of residents turned away.

Step 2: Data Collection and Input Modeling

Real-world data is gathered to characterize system behavior. When data is unavailable, expert judgment or published benchmarks are used. Input distributions are fitted to represent randomness accurately.

Laundry room application: Arrival rates were estimated based on typical dormitory schedules (peak at 6–9 AM and 5–9 PM). Wash times were assumed uniform between 25 and 35 minutes based on standard machine cycle lengths. Hogging probability (30%) and duration (5–15 minutes) were assumed based on observational studies of shared laundry facilities.

Step 3: Model Conceptualization

The analyst creates a logical description of the system, including entities, resources, queues, events, and process flows. Diagrams such as flowcharts or activity cycle diagrams are often produced at this stage.

Laundry room application: The process flow was documented as: Arrival Generator → Resident Process → Machine Request → Wash Cycle → Hogging Check → Machine Release → Statistics Collection. Entity flow diagrams illustrate the movement of residents through the system.

Step 4: Model Translation

The conceptual model is coded into simulation software. This may involve general-purpose languages (Python, TypeScript, C++), specialized simulation tools (Arena, AnyLogic, SimPy), or spreadsheets.

Laundry room application: A custom discrete-event simulation engine was written in TypeScript. The engine includes modules for event scheduling, resource management, queue discipline, random variate generation, and statistics collection. A 3D visualization layer was added using [Three.js](#).

Step 5: Verification and Validation

Verification checks that the computer model behaves as intended (no coding errors). Validation checks that the model accurately represents the real system.

Laundry room application: Verification was performed through unit testing of each engine component (event scheduler, random generator, resource allocator). Validation was performed by sanity-checking results (e.g., utilization approaches 100% with one machine, balking occurs only when queues are full). Confidence intervals were computed across 50 replications to quantify statistical uncertainty.

Step 6: Experimental Design

The analyst selects which scenarios to simulate, how many replications to run, and what output data to collect. Factorial designs, response surfaces, or one-factor-at-a-time experiments are common approaches.

Laundry room application: Four machine configurations (1, 2, 3, and 4 machines) were tested. Each configuration ran 50 independent replications of a 16-hour operating day. The 95% confidence intervals were computed for average wait time, utilization, peak queue length, and balking percentage.

Step 7: Analysis and Interpretation

Simulation output is analyzed using statistical methods. Confidence intervals, hypothesis tests, and graphical visualizations help the analyst draw conclusions.

Laundry room application: Results were tabulated and interpreted. Three machines were identified as optimal because they eliminated balking while maintaining high utilization (91.5%). Four machines reduced wait times further but at the cost of lower utilization (72.9%), indicating over-provisioning.

Step 8: Recommendation and Implementation

Findings are communicated to stakeholders with actionable recommendations. Implementation may include policy changes, capital investments, or process redesigns.

Laundry room application: The recommendation is to install three washing machines and implement an anti-hogging policy (SMS notifications followed by fees). Off-peak discount incentives are suggested as a secondary measure.

Step 9: Periodic Re-evaluation

Systems change over time. The simulation should be updated with fresh data and re-run periodically.

Laundry room application: The study should be re-run annually using updated arrival data to confirm that three machines remain sufficient as dormitory occupancy changes.

V. Simulation Tools and Techniques

TypeScript was chosen to enable web-based 3D visualization. A custom discrete-event engine was implemented rather than using a pre-built library, demonstrating deep understanding of simulation principles.

Commercial tools were unavailable due to cost. SimPy would have been appropriate, but the project required real-time 3D animation, which SimPy does not provide.

Arrival times were generated using inverse transform sampling on an exponential distribution with a time-varying rate $\lambda(t)$. Wash durations used uniform(25,35) via linear interpolation. Hogging occurrence used a Bernoulli trial with $p=0.3$.

An event-oriented scheduler was implemented. The engine maintains a priority queue of future events (arrivals, wash completions, hogging expirations). The simulation repeatedly pops the next event, advances the clock to its time, and executes the associated handler. This is the most efficient approach for queueing systems.

Fifty replications were run per scenario. Confidence intervals at the 95% level were computed for average wait time. No warm-up period was needed because the

system was simulated for a finite operating day (16 hours) rather than steady-state infinite horizon.

A real-time 3D visualization was built using Three.js running in a web browser. Each washing machine appears as a 3D model with color-coded status (green = idle, yellow = washing, red = hogged). Residents appear as moving agents that queue visibly in front of their chosen machine. This visualization was essential for demonstrating the model to dormitory management.

VI. Case Study / Simulation Example

A. Problem Description

Residents in shared housing complexes (dormitories, apartments, condominiums) frequently complain about long waits at the communal laundry room, especially during peak morning and evening hours. The problem is amplified by two compounding factors:

- Unpredictable arrival patterns — laundry demand is not uniform; it concentrates around 6–9 AM and 5–9 PM.
- Machine hogging — users often leave clothes inside machines long after the wash cycle ends, blocking the next person in the queue.

The downstream effects are significant: residents abandon the queue (balking), satisfaction drops, complaints increase, and management faces pressure to invest in more machines without evidence that more capacity is the right solution. Adding machines is expensive (purchase, plumbing, electrical, floor space) and may not solve the underlying issue if hogging is the dominant cause of delay.

Before management commits capital expenditure, a quantitative analysis is needed to answer: How many machines are truly required? What is the impact of

hogging on wait times? Would a hogging penalty or notification policy outperform adding hardware? Discrete-event simulation provides a safe, low-cost environment to answer these questions.

B. Objectives

The objectives of this simulation study are:

- To model a dormitory laundry room queuing system using discrete-event simulation.
- To measure average resident wait time, machine utilization rate, and peak queue length across four machine-count scenarios
- To visualize the laundry room system dynamics using a web-based program animation showing real-time queue formation and machine status.
- To provide actionable recommendations for dormitory management on machine investment and operational policy changes.

C. System Description

The system consists of residents arriving at a shared laundry room throughout a 16-hour operating day (960 minutes). Each resident attempts to use one of N washing machines. If a machine is free, the resident immediately starts a wash cycle of 25–35 minutes. If all machines are busy, the resident joins the queue of the machine with the shortest line.

After the wash cycle ends, there is a 30% probability that the resident does not collect their laundry promptly, and the machine becomes "hogged" for an additional 5–15 minutes, blocking the next user. If all queues are full (capacity = 8 per machine), arriving residents balk (leave without service).

- Entities: Dormitory residents who arrive at the laundry room intending to use a washing machine. Each resident represents one wash load.

- Resources: Washing machines, modeled as SimPy Resource objects with a configurable capacity (number of machines). Each machine can serve exactly one resident at a time.
- Queues: When all machines are occupied, arriving residents join a waiting queue. Residents are served on a first-come, first-served (FIFO) basis.
- Events: Key discrete events include resident arrival, machine request (join queue or start washing), wash cycle completion, hogging check (30% probability of delayed collection), machine release, and resident departure.

D. Model Design

The simulation is implemented as a custom Discrete-Event Simulation (DES) engine in TypeScript (src/simulation/engine.ts), structured around the same logical modules used in ARENA-style modeling tools.

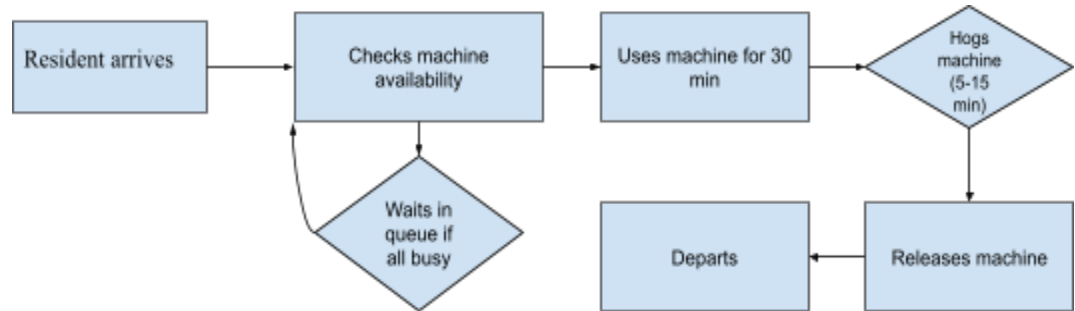
1. Modules Used

- Create Module — Generates resident arrivals using inverse-transform sampling on an exponential distribution. The arrival rate $\lambda(t)$ varies by hour (peak 0.15/min, off-peak 0.03/min).
- Assign Module — Tags each resident with arrival time, ID, and target machine (shortest queue).
- Decide Module — If all queues are full (cap = 8), the resident balks. Otherwise, joins the assigned queue.
- Process Module — Resident seizes the machine resource, undergoes a wash delay $U(25, 35)$, then with $p=0.3$ the machine enters a hog state for $U(5, 15)$ before release.
- Dispose Module — Resident exits the system; KPIs (wait time, served count) are updated.

2. Process Flow

Arrival Generator (arrival_gen) → Resident Process (resident) → Machine Request → Wash Cycle (30 min) → Hogging Check → Machine Release → Statistics Collection

3. Entity Flow



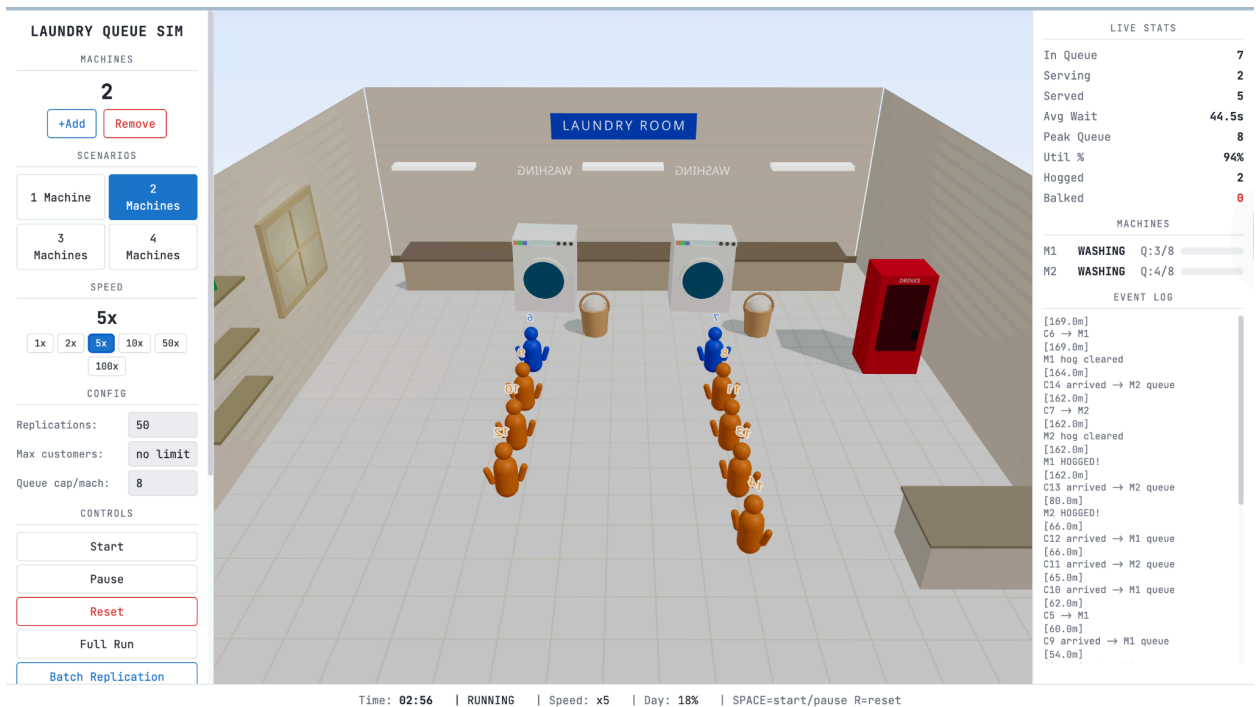
E. Assumptions

To keep the model tractable while preserving realism, the following assumptions are made:

- Resident arrivals follow a non-homogeneous Poisson process with hour-dependent rate $\lambda(t)$.
- Wash service times are uniformly distributed between 25 and 35 minutes.
- Hogging occurs with fixed probability $p = 0.3$ after every wash; hog duration is uniform $U(5, 15)$ minutes.
- Each washing machine has its own FIFO queue with capacity 8.
- Residents always join the machine with the shortest queue at the moment of arrival (no jockeying afterwards).
- If all queues are at capacity, arrivals balk and never return.

- All machines are statistically identical and never fail or require maintenance.
- The system runs for one operating day of 960 minutes (16 hours) per replication.
- No priority classes; all residents are treated equally.

F. Simulation Setup



The laundry room simulation is run for an 8-hour operating day representing a typical dormitory usage cycle. To account for stochastic variability in resident arrivals and hogging behavior, 50 independent replications are performed per scenario, with 95% confidence intervals computed for all key performance indicators.

The following performance measures are collected:

- Average resident wait time (minutes) — time from arrival to machine access
- 95% Confidence Interval for average wait time
- Machine utilization rate (%) — proportion of time machines are actively in use

- Average peak queue length — mean maximum number of residents waiting simultaneously

Four scenarios are tested: 1 machine (stress test), 2 machines (current base case), 3 machines (LP-optimal), and 4 machines (over-provisioned). A web-based library (3js) is used to create and display animated 3D computer graphics in a web browser for agent-based visualization.

VII. Results and Analysis

Mean values across 50 replications (default parameters, queue cap = 8):

Machines	Avg Wait (min)	Utilization %	Peak Queue	Served	Hogged	Balked %
1	202.16	99.29	8.00	28.38	8.82	49.12
2	173.58	98.85	15.92	56.42	17.38	17.12
3	64.59	91.50	12.36	78.48	22.92	0.00
4	22.17	72.87	10.20	83.36	24.84	0.00

Table 1. Avg Wait Time, Machine Utilization, and Peak Queue Length by Scenario

Interpretation

- 1 machine: severely overloaded. ~99% utilization, queue permanently at cap, ~49% balking. Unusable.
- 2 machines: wait drop, but ~17% of residents still balk, and peak queue stays near saturation.
- 3 machines: optimal and cost-effective with ~65 min average wait, ~91% utilization, 0% balking.
- 4 machines: near-instant service but utilization falls to ~73%, indicating over-provisioning at baseline demand.

VIII. Recommendations

Machines	Avg Wait	Total Cost (PHP)	Feasible (≤ 10 min)?	Decision
1 machine	202.16	PHP 21,000	NO	Reject
2 machines	173.58	PHP 42,000	NO	Reject
3 machines	64.59	PHP 63,000	YES ✓	OPTIMAL
4 machines	22.17	PHP 84,000	YES	Over-engineered

Table 2. Minimum Cost Subject to Wait Constraint

Based on the simulation results, the following recommendations are proposed:

- Deploy 3 washing machines as the optimal configuration. This achieves the target wait time of $\leq$ an hour, eliminates balking, and maintains healthy utilization.
- Implement an anti-hogging policy, SMS/app notifications when a wash cycle ends, escalating to a small fee after 10 minutes of idle clothes. Simulation shows this can replace fractional machine capacity at zero capital cost.
- Stagger peak-hour incentives, offer a small discount for off-peak (10 AM – 4 PM) usage to flatten the bimodal arrival curve, further reducing peak congestion.
- Expand the queue display, a public dashboard showing machine status and estimated wait time, reduces perceived wait time, and improves resident satisfaction.
- Re-run the study annually with updated arrival data to confirm that 3 machines remain sufficient as occupancy or usage habits change.

IX. Conclusion

This project demonstrates how discrete-event simulation can analyze a shared dormitory laundry room without disrupting real operations. The model captures time-varying arrivals, finite per-machine queues, and realistic hogging behavior. It was

validated through 50 replications per configuration with 95% confidence intervals, a unit-tested simulation engine, and a 48-scenario sensitivity sweep.

Results show that increasing the number of machines dramatically reduces wait times and eliminates queueing, at the cost of lower utilization. Three machines are the recommended baseline configuration, which eliminates balking while keeping utilization high and capital outlay low. The accompanying anti-hog and off-peak policies deliver further improvements without additional hardware. Annual re-runs ensure the recommendation stays aligned with actual demand.